

Objective: Analyze the CycloneDX
Authoritative Guide to:
Understand the nature of its
recommendations
Assess how test cases can be
generated

Limitation:
Only BOM Identity and Anatomy
sections reviewed so far —
classification is ongoing

Categorization:
Prescriptive Requirements
Actionable directives (e.g., must,
should, is recommended)
**Descriptive or Informational
Guidance**
Context-setting or optional
phrasing (e.g., may, can, ideally)
Underrepresentation
Important elements from the
NTIA Minimum Elements paper
missing or loosely defined in
CycloneDX

Motivation: Making concrete test cases for testing how the SBOMs created by different SBOM generation tools match with cyclonedx 1.6 documentation (the latest right now)

Related work: Multiple SBOM scoring tools already exist. What I can instead aim to do, is hyper focus on the documentation itself in order to gain a more holistic picture before going deeper. Primary sources of information for the documentation were taken from <https://cyclonedx.org/docs/1.6/json> and <https://github.com/CycloneDX/specification/blob/master/schema/bom-1.6.schema.json>

Methodology:

1. Headers: A python script created which checks which of the top headers are present, and lists down all missing ones, storing all the results in a csv
2. Validation checks: Uses the schema for validation, alongside wherever specifications are listed down for the way cyclonedx expects the value of a certain header to be, making manual checks based on that. Uses another python script that specifically checks for all these validation checks, and storing the results in a csv

Legend: 1 = Missing, 0 = Present

Field	1_syft.json	1_trivy.json	1_cyclonedx-python.json	1_cdxgen.json
bomFormat	0	0	0	0
specVersion	0	0	0	0
serialNumber	0	0	0	0
version	0	0	0	0
metadata	0	0	0	0
components	0	0	0	0
services	1	1	1	1
externalReferences	1	1	1	1
dependencies	1	0	0	0
compositions	1	1	1	1
vulnerabilities	1	0	1	1
annotations	1	1	1	0
formulation	1	1	1	1
declarations	1	1	1	1
definitions	1	1	1	1
properties	1	1	1	1
signature	1	1	1	0

Field	1_syft.json	1_trivy.json	1_cyclonedx-python.json	1_cdxgen.json
schema_valid	0	0	0	0
serialNumber_valid	0	0	0	0
version_valid	0	0	0	0
metadata_present	0	0	0	0
timestamp_valid	0	0	0	0
metadata_tools	1	1	1	1
metadata_authors	0	0	0	0
components_non_empty	0	1	0	0
unique_component_names	1	1	1	1
component_identity	1	0	0	0
purl_format	0	0	1	0
unique_purls	1	0	1	1
component_hashes	1	0	0	0
dependencies_array	0	1	0	0
dependency_refs	0	0	0	0
services_array	0	0	0	0
service_fields	0	0	0	0
vulnerabilities_array	0	0	0	0
vulnerability_entries	0	0	0	0
license_info	1	0	1	1

- schema_valid:** Validates SBOM against CycloneDX schema
- serialNumber_valid:** Checks if serialNumber follows UUID format
- version_valid:** Ensures SBOM version is an integer ≥ 1
- metadata_present:** Checks if metadata block exists
- timestamp_valid:** Validates metadata timestamp format
- metadata_tools:** Ensures tools have valid name and version
- metadata_authors:** Checks if authors list is properly structured
- components_non_empty:** Verifies components list is not empty
- unique_component_names:** Ensures all component names are unique
- component_identity:** Each component must have a purl or cpe
- purl_format:** Validates format of purl fields
- unique_purls:** Checks purls are unique across components
- component_hashes:** Ensures each component has non-empty list of hashes
- dependencies_array:** Checks dependencies field is a list or missing
- dependency_refs:** Validates dependency references match component IDs
- services_array:** Ensures services field is a list or missing
- service_fields:** Each service must have name and endpoint
- vulnerabilities_array:** Ensures vulnerabilities field is a list or missing
- vulnerability_entries:** Each vulnerability must have an ID and ratings list
- license_info:** At least one license must be present in metadata or components

Key Specification Points (from CycloneDX 1.6):

- Graph Structure: SBOMs "SHOULD ideally contain all direct and transitive components and their dependency relationships." Graph "SHOULD be one node deep."
- Leaf Nodes: "Components that do not have dependencies MUST be declared as empty elements within the graph" (i.e., dependsOn: []).
- Root Component: SBOM metadata should define the component the BOM describes (Guide p.12).
- Component Identity: Components identified via bom-ref (unique within BOM), ideally PURL/UUID .

Test Cases for Dependency Requirements

Selected Test Cases & Expected Outcomes (from our Audit Table):

- TC-DEP.ROOT: Verify correct metadata.component for the project.
Expected: metadata.component clearly identifies the project being analyzed.
- TC-DEP.DIRECT_GRAPH: Verify direct dependencies linked to root & flat graph.
Expected: dependencies section links root to direct deps; graph is one node deep.
- TC-DEP.LEAF: Verify leaf nodes have dependsOn: [].
Expected: Components with no further dependencies have { "ref": "...", "dependsOn": [] }.
- TC-DEP.TRANS_RAW: Verify transitive dependency resolution from raw manifests (e.g., requirements.txt).
Expected (Ideal): Full transitive chain resolved and represented.
- TC-DEP.TRANS_LOCK: Verify transitive dependency resolution from lockfiles (e.g., poetry.lock).
Expected: Full transitive chain resolved and represented accurately based on lockfile.

Initial Tool Audit Results & Next Steps

Testbed was minimal Python project with requirements.txt (six==1.16.0, tinyrecord==0.2.0). No lockfile for initial tests.

Tool Execution & Key Observations

1. Syft (v1.24.0):

- Identified packages (six, tinyrecord) as components.
- Divergence: metadata.component generic; dependencies graph entirely MISSING.

2. cyclonedx-python (v6.1.1):

- Identified packages (six, tinyrecord) as components.
- Divergence: metadata.component MISSING; dependencies graph incomplete (lists deps as roots, misses dependsOn: [] for leaves).

Preliminary Insights:

- Insight 1: Tools show varied ability/approaches to defining the project's root component from minimal source.
- Insight 2: Significant divergence in generating the dependencies graph structure even for simple direct dependencies from requirements.txt. Adherence to spec for leaf nodes is inconsistent.
- Insight 3 (Underspecification?): The spec's "SHOULD ideally" for transitive dependency completeness might allow tools flexibility that leads to these varied (and often incomplete from user perspective) outputs when parsing raw manifests.